Radix Sort

AlgoExpert · Hard · Sorting

The problem

- Sort non-negative ints by LSD radix sort with counting sort per decimal digit.

Intuition

- Sort by each decimal digit from least significant, using a stable counting sort each time.

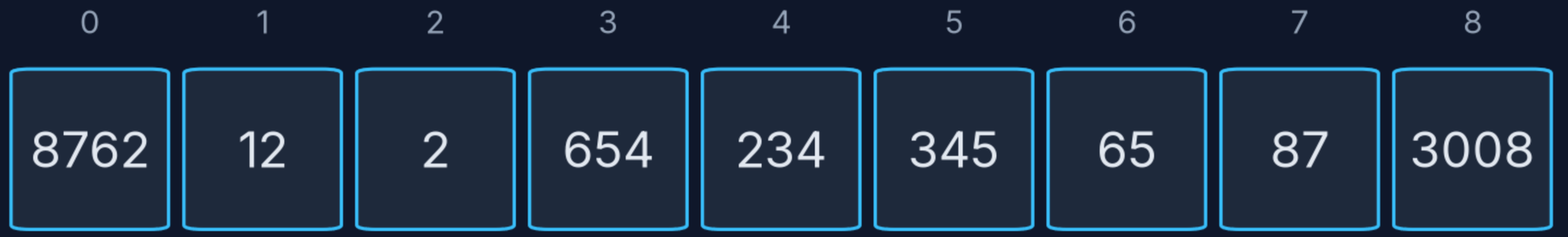
Brute force vs optimal

- Comparison sorts → $\Omega(n \log n)$
- Radix sort with counting sort per digit → $O(d \cdot (n + 10))$
- Requires stable per-digit sort

Approach

- 1. Find the max to know digit count
- 2. For each digit, count, prefix sum, and place stably
- 3. Copy back

Worked example



`stable counting sort by digit 0 (×1)`

Sort by the ones digit

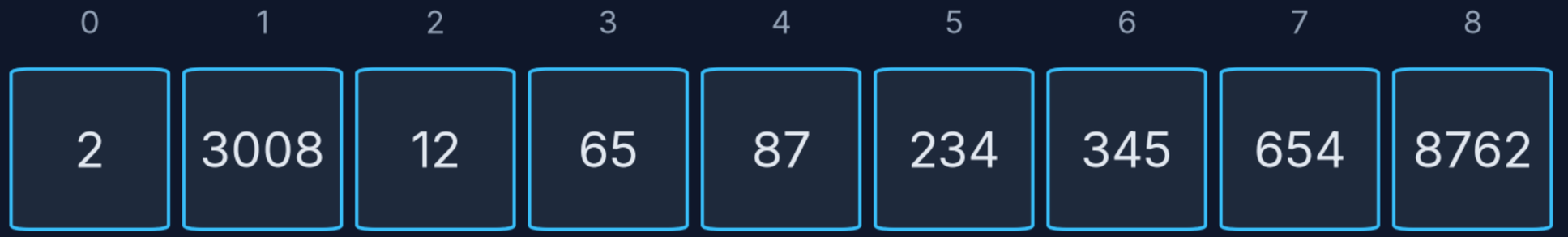
Worked example



`stable counting sort by digit 1 (×10)`

Sort by the tens digit

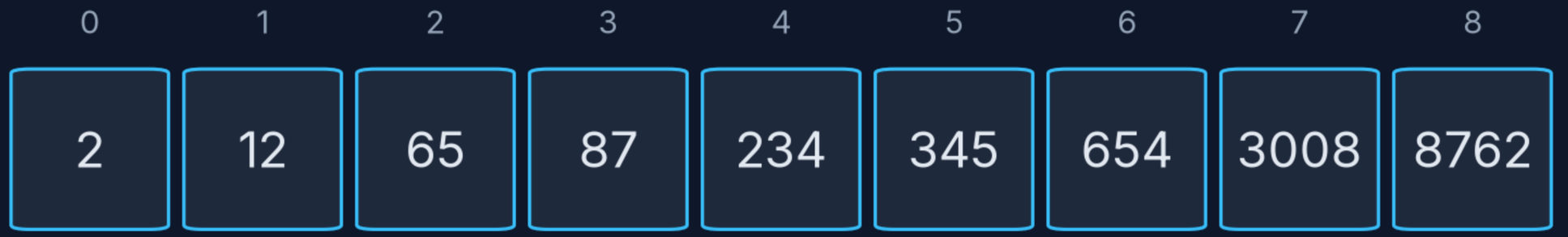
Worked example



`stable counting sort by digit 2 (×100)`

Sort by the hundreds digit

Worked example



`stable counting sort by digit 3 (×1000)`

Sort by the thousands digit

Java solution

```java
public class RadixSort {
    public static ArrayList<Integer> radixSort(ArrayList<Integer> a) {
        if (a.isEmpty()) return a;
        int max = a.stream().max(Integer::compare).get();
        int[] out = new int[a.size()];
        for (long exp = 1; max / exp > 0; exp *= 10) {
            int[] count = new int[10];
            for (int x : a) count[(int) (x / exp % 10)]++;
            for (int i = 1; i < 10; i++) count[i] += count[i - 1];
            for (int i = a.size() - 1; i >= 0; i--) out[--count[(int) (a.get(i) / exp % 10)]] = a.get(i);
            for (int i = 0; i < out.length; i++) a.set(i, out[i]);
        }
        return a;
    }
}
```

Complexity

- Time: O(d * (n + 10))
- Space: O(n)

Edge cases & pitfalls

- Stability is essential
- Iterate counts from the end
- Negative numbers need special handling

Interview tips & follow-ups

- When radix beats comparison sorts

Recap

- Sort by each decimal digit from least significant, using a stable counting sort each time.
- Complexity: O(d * (n + 10))